```python
import nltk
import numpy as np
from collections import Counter, defaultdict
from nltk.corpus import reuters
from nltk.util import ngrams
nltk.download('reuters')
nltk.download('punkt')
```

```
[nltk_data] Downloading package reuters to /root/nltk_data...
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
True
```

```python
words = reuters.words()


tokens = [w.lower() for w in words if w.isalpha()]

split_index = int(0.8 * len(tokens))
train_tokens = tokens[:split_index]
test_tokens = tokens[split_index:]
```

```python
unigram_counts = Counter(train_tokens)
total_unigrams = sum(unigram_counts.values())

unigram_probs = {word: count / total_unigrams
                 for word, count in unigram_counts.items()}
```

```python
def compute_entropy(prob_dist):
    entropy = -sum(p * np.log2(p) for p in prob_dist.values())
    return entropy

entropy = compute_entropy(unigram_probs)
print("Entropy (Unigram Model):", entropy)
```

```
Entropy (Unigram Model): 9.961577539326788
```

```python
def compute_cross_entropy(test_tokens, model_probs, vocab_size):
    N = len(test_tokens)
    cross_entropy = 0

    for word in test_tokens:

        prob = model_probs.get(word, 1 / (total_unigrams + vocab_size))
        cross_entropy += -np.log2(prob)

    return cross_entropy / N

vocab_size = len(unigram_counts)
cross_entropy = compute_cross_entropy(test_tokens, unigram_probs, vocab_size)
print("Cross-Entropy:", cross_entropy)
```

```
Cross-Entropy: 10.002042244862325
```

```python
perplexity = 2 ** cross_entropy
print("Perplexity:", perplexity)
```

```
Perplexity: 1025.4505765574202
```